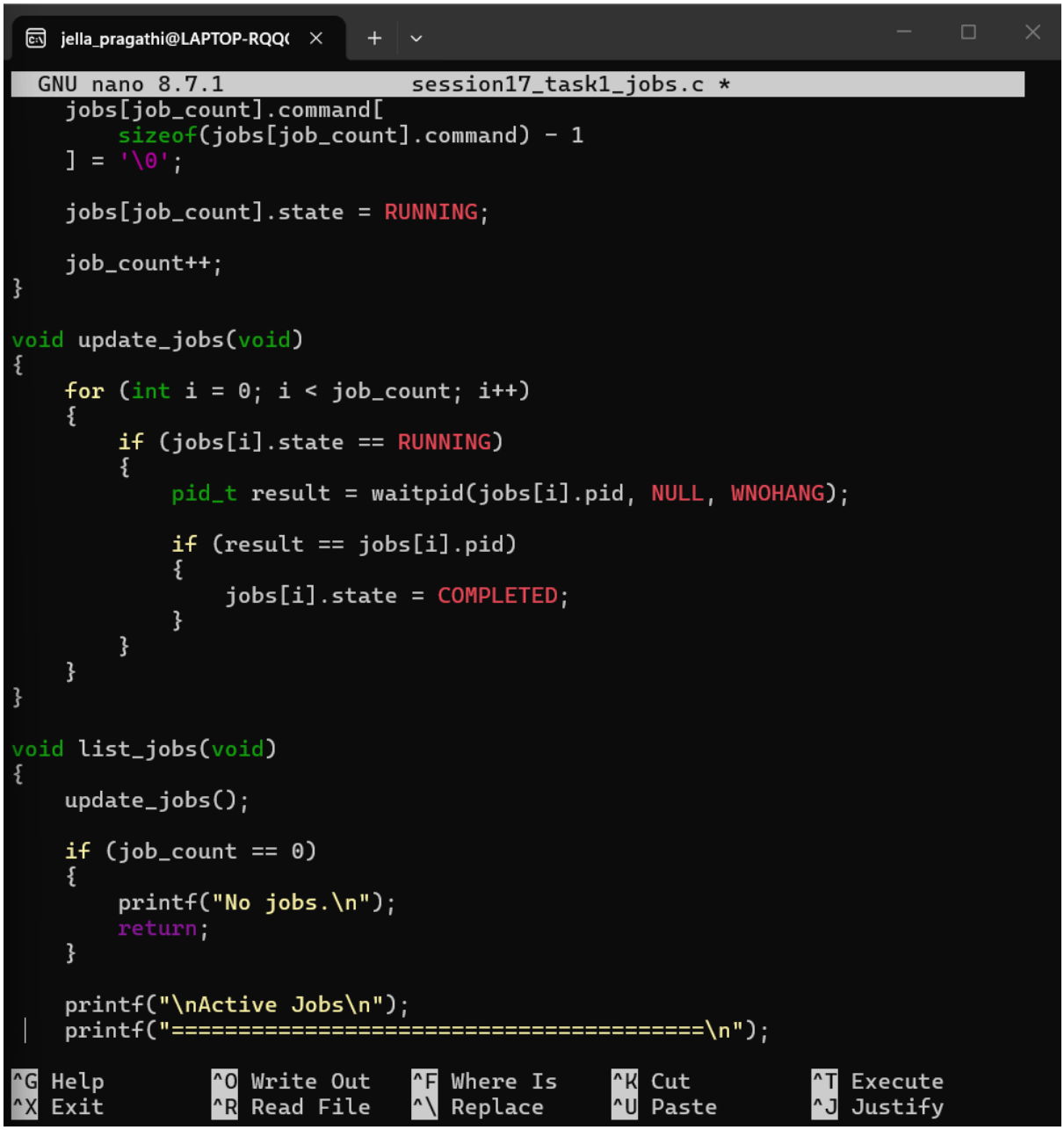
Skill Week 9

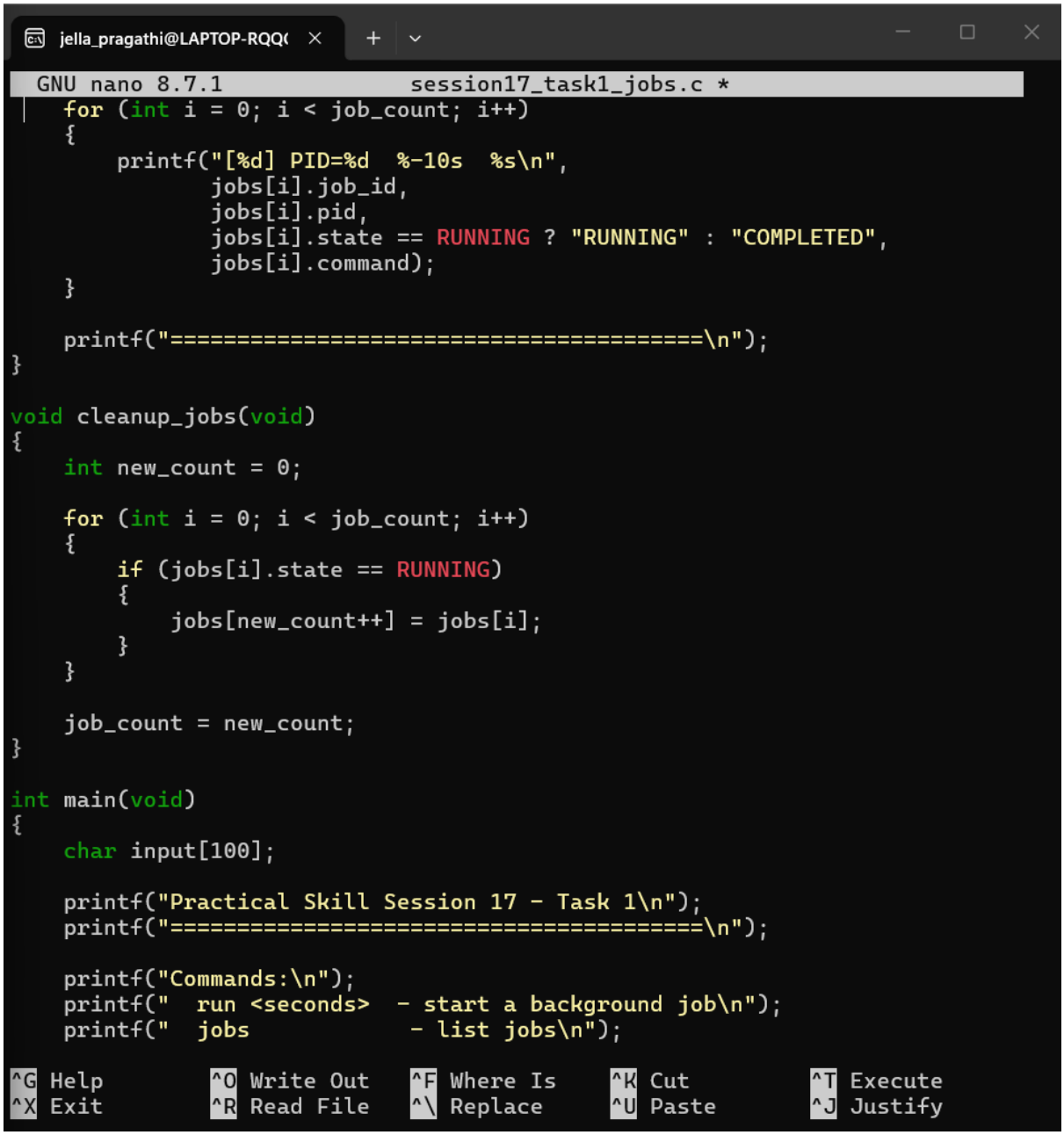
Session 17

Task 1

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

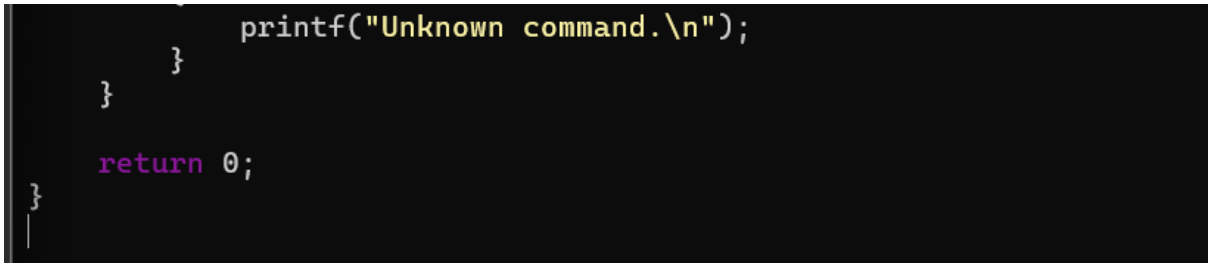


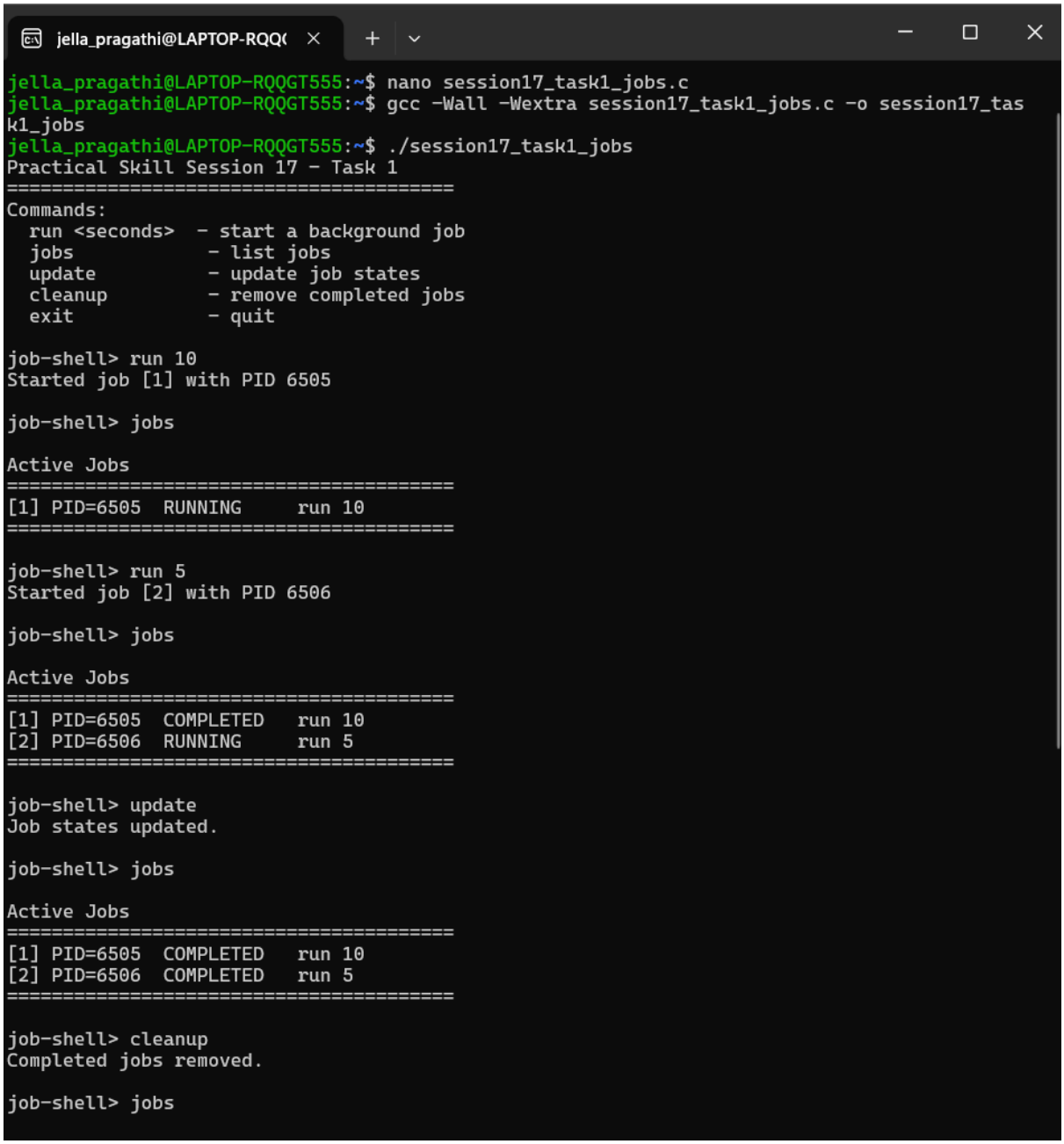

jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task1_jobs.c *
jobs[job_count].command[
sizeof(jobs[job_count].command) - 1
] = '\0';
jobs[job_count].state = RUNNING;
job_count++;
}
void update_jobs(void)
{
for (int i = 0; i < job_count; i++)
{
if (jobs[i].state == RUNNING)
{
pid_t result = waitpid(jobs[i].pid, NULL, WNOHANG);
if (result == jobs[i].pid)
{
jobs[i].state = COMPLETED;
}
}
}
}
void list_jobs(void)
{
update_jobs();
if (job_count == 0)
{
printf("No jobs.\n");
return;
}
printf("\nActive Jobs\n");
printf("===\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task1_jobs.c *
for (int i = 0; i < job_count; i++)
{
printf("[%d] PID=%d %-10s %s\n",
jobs[i].job_id,
jobs[i].pid,
jobs[i].state == RUNNING ? "RUNNING" : "COMPLETED",
jobs[i].command);
}
printf("==\n");
}
void cleanup_jobs(void)
{
int new_count = 0;
for (int i = 0; i < job_count; i++)
{
if (jobs[i].state == RUNNING)
{
jobs[new_count++] = jobs[i];
}
}
job_count = new_count;
}
int main(void)
{
char input[100];
printf("Practical Skill Session 17 - Task 1\n");
printf("==\n");
printf("Commands:\n");
printf(" run <seconds> - start a background job\n");
printf(" jobs - list jobs\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


jella_pragathi@LAPTOP-RQQ(
Default: jella_pragathi@LAPTOP-RQQGT555: ~
ctrl+alt+1
ession17_task1_jobs.c *
update job states\n");
printf(" cleanup - remove completed jobs\n");
printf(" exit - quit\n");
while (1)
{
printf("\njob-shell> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
{
break;
}
input[strcspn(input, "\n")] = '\0';
if (strcmp(input, "exit") == 0)
{
break;
}
if (strcmp(input, "jobs") == 0)
{
list_jobs();
}
else if (strcmp(input, "update") == 0)
{
update_jobs();
printf("Job states updated.\n");
}
else if (strcmp(input, "cleanup") == 0)
{
cleanup_jobs();
printf("Completed jobs removed.\n");
}
else if (strncmp(input, "run ", 4) == 0)
{
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute

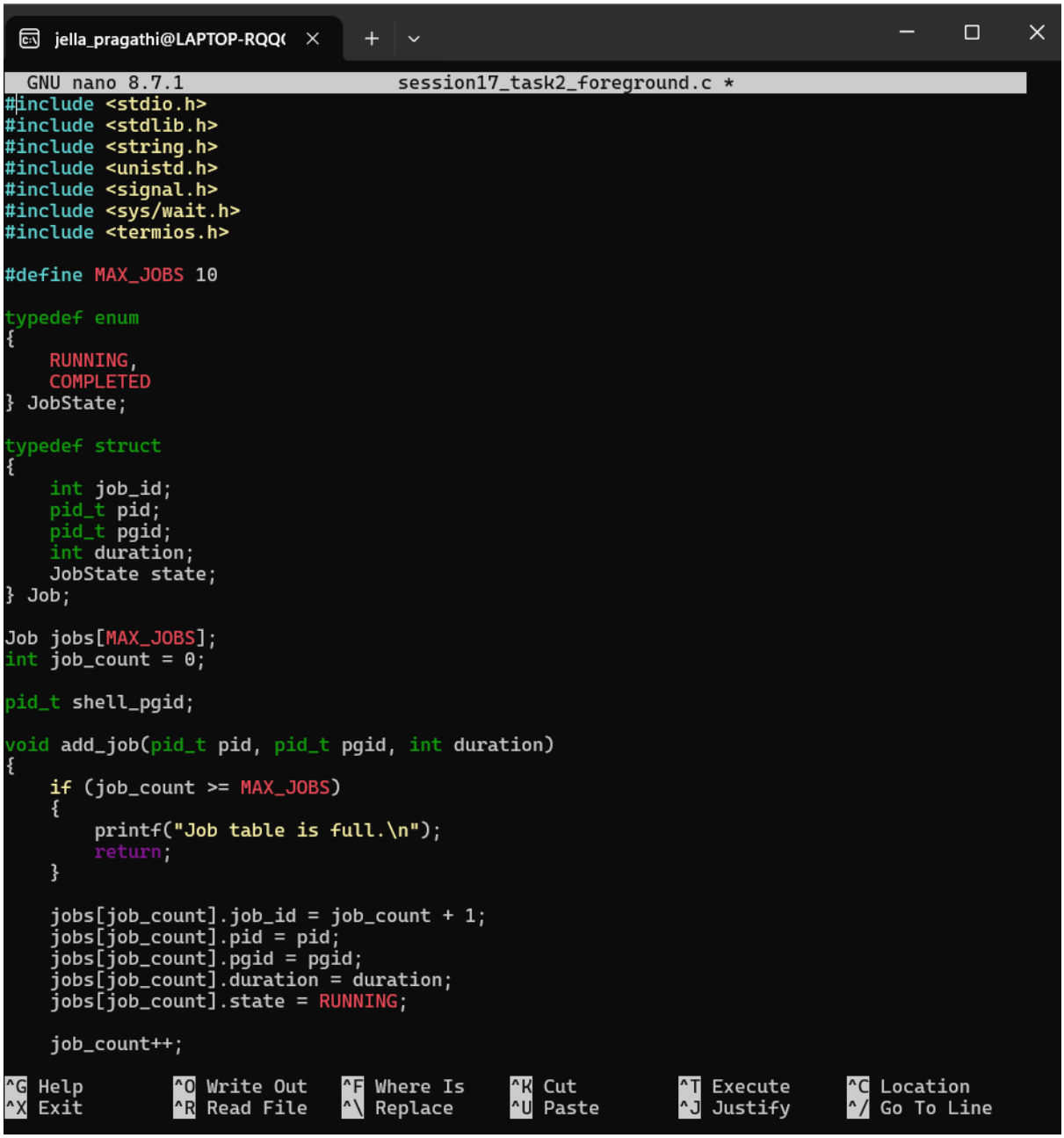

jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task1_jobs.c *
printf("Completed jobs removed.\n");
}
else if (strncmp(input, "run ", 4) == 0)
{
int seconds = atoi(input + 4);
if (seconds <= 0)
{
printf("Invalid duration.\n");
continue;
}
pid_t pid = fork();
if (pid < 0)
{
perror("fork");
continue;
}
if (pid == 0)
{
sleep(seconds);
exit(0);
}
add_job(pid, input);
printf("Started job [%d] with PID %d\n",
job_count,
pid);
}
else
{
printf("Unknown command.\n");
}
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


printf("Unknown command.\n");
}
}
return 0;
}


jella_pragathi@LAPTOP-RQQ(
jella_pragathi@LAPTOP-RQQGT555:~$ nano session17_task1_jobs.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra session17_task1_jobs.c -o session17_tas
k1_jobs
jella_pragathi@LAPTOP-RQQGT555:~$ ./session17_task1_jobs
Practical Skill Session 17 - Task 1
Commands:
run <seconds> - start a background job
jobs - list jobs
update - update job states
cleanup - remove completed jobs
exit - quit
job-shell> run 10
Started job [1] with PID 6505
job-shell> jobs
Active Jobs
[1] PID=6505 RUNNING run 10
job-shell> run 5
Started job [2] with PID 6506
job-shell> jobs
Active Jobs
[1] PID=6505 COMPLETED run 10
[2] PID=6506 RUNNING run 5
job-shell> update
Job states updated.
job-shell> jobs
Active Jobs
[1] PID=6505 COMPLETED run 10
[2] PID=6506 COMPLETED run 5
job-shell> cleanup
Completed jobs removed.
job-shell> jobs

Task 2

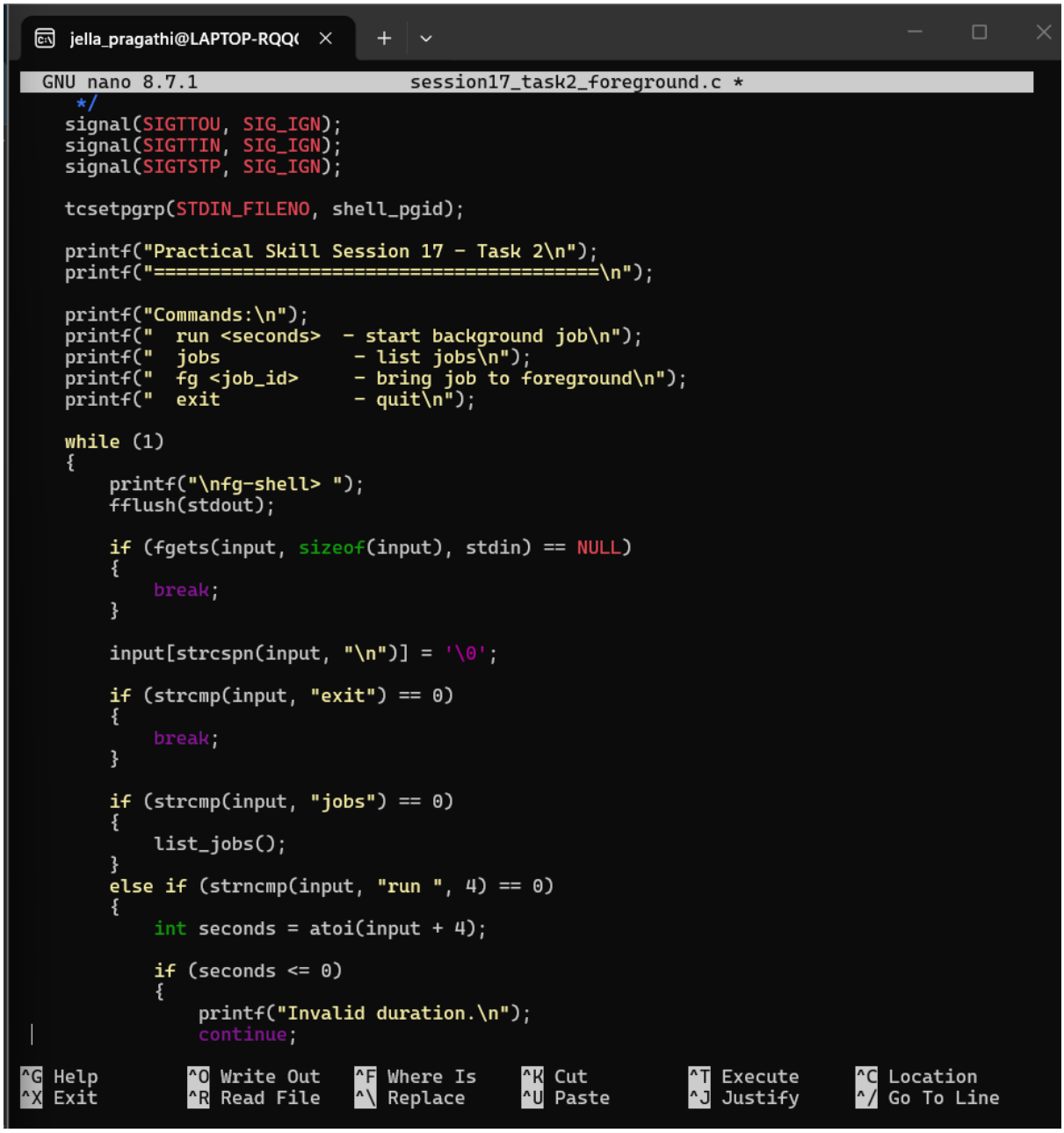
To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching



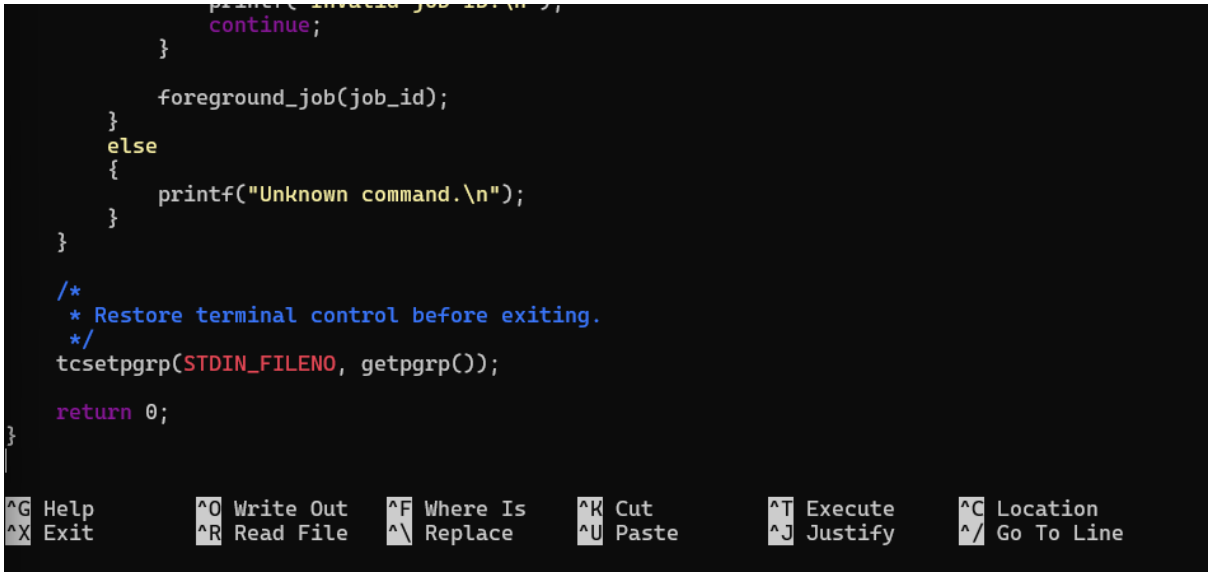

jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task2_foreground.c *
}
void update_jobs(void)
{
for (int i = 0; i < job_count; i++)
{
if (jobs[i].state == RUNNING)
{
pid_t result = waitpid(
jobs[i].pid,
NULL,
WNOHANG
);
if (result == jobs[i].pid)
{
jobs[i].state = COMPLETED;
}
}
}
}
void list_jobs(void)
{
update_jobs();
if (job_count == 0)
{
printf("No jobs.\n");
return;
}
printf("\nJob List\n");
printf("==\n");
for (int i = 0; i < job_count; i++)
{
printf("[%d] PID=%d PGID=%d %-10s duration=%d sec\n",
jobs[i].job_id,
jobs[i].pid,
jobs[i].pgid,
jobs[i].state == RUNNING
? "RUNNING"
: "COMPLETED",
jobs[i].duration);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task2_foreground.c *
}
printf("==\n");
}
void foreground_job(int job_id)
{
update_jobs();
int index = -1;
for (int i = 0; i < job_count; i++)
{
if (jobs[i].job_id == job_id)
{
index = i;
break;
}
}
if (index == -1)
{
printf("Job [%d] not found.\n", job_id);
return;
}
if (jobs[index].state == COMPLETED)
{
printf("Job [%d] is already completed.\n", job_id);
return;
}
pid_t pgid = jobs[index].pgid;
printf("\nBringing job [%d] to foreground...\n", job_id);
printf("Transferring terminal control to PGID %d\n", pgid);
if (tcsetpgrp(STDIN_FILENO, pgid) == -1)
{
perror("tcsetpgrp");
return;
}
printf("Sending SIGCONT to job [%d]\n", job_id);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task2_foreground.c *
if (kill(-pgid, SIGCONT) == -1)
{
perror("kill");
}
printf("Waiting for foreground job to finish...\n");
int status;
if (waitpid(
jobs[index].pid,
&status,
0) == -1)
{
perror("waitpid");
}
jobs[index].state = COMPLETED;
printf("Job [%d] completed.\n", job_id);
printf("Restoring terminal control to shell...\n");
if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
{
perror("tcsetpgrp");
}
}
int main(void)
{
char input[100];
shell_pgid = getpid();
/*
* Put this shell into its own process group.
*/
setpgid(0, shell_pgid);
/*
* Ignore terminal job-control signals in the shell.
*/
signal(SIGTTOU, SIG_IGN);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task2_foreground.c *
*/
signal(SIGTTOU, SIG_IGN);
signal(SIGTTIN, SIG_IGN);
signal(SIGTSTP, SIG_IGN);
tcsetpgrp(STDIN_FILENO, shell_pgid);
printf("Practical Skill Session 17 - Task 2\n");
printf("==\n");
printf("Commands:\n");
printf(" run <seconds> - start background job\n");
printf(" jobs - list jobs\n");
printf(" fg <job_id> - bring job to foreground\n");
printf(" exit - quit\n");
while (1)
{
printf("\nfg-shell> ");
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
{
break;
}
input[strcspn(input, "\n")] = '\0';
if (strcmp(input, "exit") == 0)
{
break;
}
if (strcmp(input, "jobs") == 0)
{
list_jobs();
}
else if (strncmp(input, "run ", 4) == 0)
{
int seconds = atoi(input + 4);
if (seconds <= 0)
{
printf("Invalid duration.\n");
continue;
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


jella_pragathi@LAPTOP-RQQ(
GNU nano 8.7.1 session17_task2_foreground.c *
pid_t pid = fork();
if (pid < 0)
{
perror("fork");
continue;
}
if (pid == 0)
{
/*
* Child becomes a new process group leader.
*/
setpgid(0, 0);
printf("Background job running for %d seconds...\n",
seconds);
sleep(seconds);
printf("Background job finished.\n");
exit(0);
}
/*
* Parent places child in its own process group.
*/
setpgid(pid, pid);
add_job(pid, pid, seconds);
printf("Started job [%d] PID=%d PGID=%d\n",
job_count,
pid,
pid);
}
else if (strncmp(input, "fg ", 3) == 0)
{
int job_id = atoi(input + 3);
if (job_id <= 0)
{
printf("Invalid job ID.\n");
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line



Skill Session 18

Task 1

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

Task 2

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery